AGENCYOS

AI Orchestrator & Multi-Model Routing Specification

Document 04 of the AgencyOS Master Documentation Set

GEMINI • OPENAI/CHATGPT • CLAUDE • GROK • DEEPSEEK • FUTURE PROVIDERS

Purpose: Define how AgencyOS selects, routes, evaluates, falls back between, and governs multiple AI providers without locking the business to any single model vendor.

1. Core Objective

> AgencyOS must be model-agnostic: the business defines the task and required capability; the Orchestrator decides which available AI provider/model should perform it.

• The system must support multiple providers simultaneously.

• Different agents may use different providers.

• The same agent may switch providers based on task type, quality, cost, latency or availability.

• A provider outage must not automatically stop the agency if an approved fallback exists.

• Model choice must never override business policy, security permissions or Admin authority.

• Provider credentials belong to the Admin-controlled Integrations system, not individual agents.

2. Supported Provider Layer

Initial provider targets:

• Google Gemini — multimodal reasoning, long-context tasks, UI/image understanding and general agent work.

• OpenAI / ChatGPT models — general reasoning, structured workflows, coding/tool use and agentic tasks.

• Anthropic / Claude — long-context reasoning, document-heavy workflows, coding and complex agent tasks.

• xAI / Grok — general reasoning, coding and tasks that benefit from its supported modalities/search capabilities.

• DeepSeek — cost-efficient reasoning/coding and high-volume engineering workloads.

• Future providers — any provider implementing the internal AgencyOS provider contract.

Provider/model availability changes over time. Therefore the implementation must discover or configure currently available models rather than hard-code a permanent model list. For example, current provider documentation exposes dynamic model lists and model metadata; DeepSeek currently documents V4-Pro/V4-Flash, while xAI exposes model-list APIs and current model aliases. ■cite■turn0search0■turn0search1■turn0search5■

3. Important Design Decision — Capability Tiers, Not Permanent Model Names

> Do not write business logic such as 'Sales Agent always uses Model X forever.' Write 'Sales Agent requires SALES_HIGH_QUALITY capability, with preferred provider/model and fallback order configured by Admin.'

• Provider/model IDs are configuration.

• Agent capability requirements are product logic.

• The Router maps requirements to currently available models.

• This protects AgencyOS when a provider retires, renames or replaces a model.

• Use provider aliases or pinned versions according to Admin policy. Pinned versions are preferred for workflows requiring deterministic behavior; latest aliases may be used where continuous improvement is acceptable.

This matters in practice: provider model catalogs and legacy names change. DeepSeek, for example, has deprecated older model names in favor of its V4 family, while xAI documents aliases and versioned model identifiers. ■cite■turn0search0■turn0search2■turn0search6■

4. Recommended Agent-to-Provider Routing Matrix

Agent / Workload	Primary	Fallback 1	Fallback 2	Routing priority
Sales Agent	OpenAI or Claude	Gemini	Grok	Conversation quality + reliability + cost

Trust/Negotiation	Claude or OpenAI	Gemini	Grok	Reasoning + tone + policy compliance
Project Manager	Claude or OpenAI	Gemini	DeepSeek	Long context + structured planning
UI Designer	Gemini	OpenAI	Claude	Multimodal/UI understanding + generation
Prototype Agent	OpenAI	Claude	Gemini	Tool use + code + structured output
Frontend Developer	Claude / OpenAI	DeepSeek	Gemini	Coding quality + tool use + cost
Backend Developer	Claude / OpenAI	DeepSeek	Gemini	Reasoning + coding + reliability
Database Developer	Claude / OpenAI	DeepSeek	Gemini	Schema/reasoning + correctness
DevOps Agent	OpenAI / Claude	DeepSeek	Gemini	Tool use + deterministic execution
QA Agent	Claude / OpenAI	Gemini	DeepSeek	Deep reasoning + test analysis
Finance Agent	OpenAI / Claude	Gemini	DeepSeek	Structured output + correctness
Support Agent	Fast OpenAI/Gemini tier	Claude	Grok	Latency + cost + tone
Customer Success	OpenAI/Gemini	Claude	Grok	Conversation + cost
Upsell Agent	OpenAI/Claude	Gemini	Grok	Sales reasoning + policy adherence
Orchestrator	OpenAI/Claude	Gemini	DeepSeek	Routing/reasoning + structured decisions

This matrix is the recommended starting policy, not an immutable technical rule. Admin must be able to change provider/model assignments without changing agent code.

5. Routing Decision Pipeline

> TASK → CAPABILITY REQUIREMENTS → POLICY CHECK → CANDIDATE MODELS → HEALTH CHECK → COST CHECK → QUALITY TIER → ROUTE → EXECUTE → VALIDATE → FALLBACK/ESCALATE

• Step 1: Identify task and responsible agent.

• Step 2: Determine required capabilities: text, image, audio, code, reasoning, structured JSON, long context, tool use, etc.

• Step 3: Apply Admin routing policy.

• Step 4: Remove unavailable, disabled or unauthorized providers.

• Step 5: Check current provider health and rate limits.

• Step 6: Estimate cost.

• Step 7: Select the best eligible model.

• Step 8: Execute with timeout/retry policy.

• Step 9: Validate output.

• Step 10: If validation fails, use approved fallback or escalate.

6. Capability Profiles

• GENERAL_FAST — routine chat, simple classification, reminders, basic support.

• GENERAL_HIGH_QUALITY — complex client communication and business reasoning.

• REASONING_HIGH — complex planning, analysis, QA and difficult debugging.

• CODING_HIGH — multi-file code generation, debugging, refactoring and architecture.

• CODING_COST_OPTIMIZED — high-volume coding tasks where cost matters.

• MULTIMODAL — image/screenshot/UI/design understanding.

• LONG_CONTEXT — large project documents, requirements and repository context.

• STRUCTURED_OUTPUT — strict JSON/schema-constrained workflows.

• TOOL_AGENT — reliable function/tool calling and multi-step execution.

• REALTIME_AWARE — workflows requiring approved live/search capabilities.

• EMBEDDING/RETRIEVAL — semantic memory and document retrieval, where supported.

7. Model Selection Score

The Router should compute an eligibility/selection score rather than blindly choosing the cheapest or most powerful model.

> ROUTE_SCORE = Quality × PolicyFit × Availability × Reliability × CapabilityFit ÷ (Cost × LatencyPenalty)

• Quality score comes from controlled internal evaluations, not marketing claims alone.

• CapabilityFit checks whether the model actually supports required modality/tool/schema/context requirements.

• PolicyFit checks Admin restrictions and per-agent rules.

• Availability includes provider health, rate limits and quota.

• Reliability includes recent successful executions and validation outcomes.

• Cost uses current provider pricing/configuration.

• Latency matters for client-facing conversations.

The exact formula and weights should be configurable and versioned so routing behavior can evolve without rewriting agent code.

8. Cost Management

• Admin can set global daily/monthly AI budget.

• Admin can set per-provider budget.

• Admin can set per-agent budget.

• Admin can set per-project AI budget.

• Admin can set per-task maximum spend where supported.

• Use cheaper models for simple tasks and premium models for high-impact tasks.

• Cache/reuse stable context where provider/API support permits.

• Avoid sending unnecessary project history to every model call.

• Track input/output tokens and provider-reported usage where available.

• Trigger alerts when budget thresholds are reached.

• When a budget threshold is exceeded, route to a lower-cost approved tier or require Admin approval.

9. Quality Management

Quality must be measured internally

• Create a benchmark suite for each agent role.

• Run representative sales conversations.

• Run objection-handling tests.

• Run requirement-extraction tests.

• Run code-generation tasks with known expected outcomes.

• Run UI-analysis tasks.

• Run QA/test-generation tasks.

• Run structured JSON output tests.

• Run long-context retrieval tests.

• Track failure rate, hallucination rate, tool-call errors and validation failures.

Provider marketing claims should not be used as the sole routing criterion. AgencyOS should maintain its own evaluation scores.

10. Availability & Health

• Each provider integration has ENABLED/DISABLED status.

• Run controlled health checks.

• Record latency and error rates.

• Detect rate-limit responses.

• Detect quota exhaustion.

• Detect authentication failures.

• Detect provider outages/timeouts.

• Remove unhealthy models temporarily from candidate selection.

• Automatically restore candidates after successful health checks.

• Admin can manually disable a provider/model.

11. Fallback Strategy

> Primary failure must not equal workflow failure.

• Timeout → retry according to safe policy.

• Transient provider error → retry or alternate provider.

• Rate limit → route to next eligible provider.

• Quota exhaustion → route to next eligible provider or notify Admin.

• Schema/output validation failure → retry with correction or alternate model.

• Tool failure → retry only when operation is safe/idempotent.

• Repeated failure → escalate to Orchestrator/Admin.

• Never repeat a non-idempotent financial/destructive action blindly.

12. Fallback Chains by Agent

Agent	Example fallback chain
Sales	OpenAI/Claude → Gemini → Grok → Admin escalation
PM	Claude/OpenAI → Gemini → DeepSeek → Admin escalation
UI	Gemini → OpenAI → Claude → Admin/design review
Prototype	OpenAI → Claude → Gemini → Developer escalation
Development	Claude/OpenAI → DeepSeek → Gemini → Developer escalation
QA	Claude/OpenAI → Gemini → DeepSeek → human review
Finance	OpenAI/Claude → Gemini → Admin review
Support	Fast OpenAI/Gemini → Claude → Admin/support escalation
Customer Success	OpenAI/Gemini → Claude → Admin
Upsell	OpenAI/Claude → Gemini → Admin
Orchestrator	Primary reasoning model → second independent provider → Admin

13. Admin AI Routing Control Center

• Provider cards: OpenAI, Anthropic, Google, xAI, DeepSeek and future providers.

• Secure API-key configuration.

• Enable/disable provider.

• Test connection.

• Health status.

• Available models discovered from provider or configured manually.

• Preferred model.

• Fallback order.

• Agent-to-provider assignment.

• Capability mapping.

• Cost limits.

• Latency limits.

• Daily/monthly usage.

• Budget alerts.

• Routing logs.

• Manual override.

• Emergency provider disable.

• Configuration version/history.

API keys must be encrypted at rest, masked in the UI, never exposed to agents unnecessarily, and never included in client-facing messages.

14. Dynamic Model Discovery

The Admin integration layer should support model discovery when the provider exposes it. xAI, for example, documents model-list APIs that return model IDs, aliases, context and pricing metadata. ■cite■turn0search1■turn0search4■

• Store provider model ID, display name, capabilities, context limit, modalities and pricing metadata where available.

• Store when metadata was last refreshed.

• Do not automatically activate a newly discovered model without an Admin policy or safe-default rule.

• Allow Admin to mark models as approved, blocked, experimental or fallback-only.

15. Model Versioning Strategy

• LATEST — follows provider's latest alias; useful for low-risk/experimental workflows.

• PINNED — exact model version; preferred for deterministic production workflows.

• CANARY — limited traffic for evaluation before wider activation.

• DEPRECATED — no new tasks; used only for migration where necessary.

• BLOCKED — never route.

Model migration must be a controlled configuration change, not an emergency code rewrite.

16. Prompt & Context Portability

• Agent prompts must be provider-neutral wherever possible.

• Provider-specific adapters may translate system prompts, tool schemas and structured-output requirements.

• Do not embed provider-specific assumptions in core business logic.

• Normalize responses into an internal AgencyOS response contract.

• Normalize tool calls into an internal tool contract.

• Preserve the original provider response for audit/debugging when policy permits.

• Sensitive client information must be minimized before cross-provider routing.

17. Structured Output Contract

For workflow-critical tasks, the Router should prefer models/providers that support reliable structured output.

• Quotation extraction → structured quotation object.

• Requirement extraction → structured requirement schema.

• Agent handoff → task schema.

• Approval request → approval schema.

• QA result → test/result schema.

• Finance result → invoice/payment schema.

• Use JSON/schema validation before accepting model output.

DeepSeek currently documents JSON output support, illustrating why provider capability metadata should be represented in the routing layer. ■cite■turn0search8■

18. Security & Data Routing

• Provider selection must respect data-classification policy.

• Highly sensitive data may be restricted to approved providers.

• Client credentials, API keys and secrets must never be passed to general-purpose models unless explicitly required by a secure tool.

• Use redaction/minimization for unnecessary PII.

• Keep organization/project context isolated.

• Log which provider received a task, subject to the agency's privacy policy.

• Admin can block specific providers for specific agents or data classes.

19. Cost vs Quality Routing Examples

Example A — New WhatsApp lead

Use a fast/low-cost model for greeting, classification and simple follow-up. Escalate to a higher-quality model when the conversation becomes complex or commercial.

Example B — High-value negotiation

Use a high-quality reasoning/conversation model. Do not downgrade purely for cost when the task could materially affect revenue.

Example C — Bulk coding

Use a cost-efficient coding model for routine repetitive implementation; route architecture/debugging to a higher-quality coding/reasoning model.

Example D — QA

Use a strong reasoning model for difficult failure analysis and a cheaper model for routine test-case generation, with validation.

Example E — Support FAQ

Use a fast inexpensive model against approved knowledge; escalate uncertain cases to a stronger model or human/Admin.

20. Agent Independence

> Agents should depend on capability contracts, not on one provider.

• Sales Agent can continue if its preferred provider is unavailable.

• Developer Agent can switch providers between tasks.

• QA can independently review code produced by a different provider.

• This creates model diversity and reduces correlated failure.

• Where practical, critical QA/review should use a different provider/model family from the one that produced the artifact.

21. Multi-Model Review for Critical Tasks

• For high-value or high-risk tasks, AgencyOS may use a primary model plus an independent reviewer.

• Example: Developer A produces code → QA/reviewer B analyzes it.

• Example: Sales Agent prepares high-value quotation → policy checker/reviewer validates it.

• Example: Finance Agent prepares invoice → deterministic application validation checks totals.

• The reviewer does not replace deterministic validation.

Use multi-model review selectively because it increases cost and latency.

22. Observability & Routing Logs

• Record task ID, agent, provider, model, routing reason, capability requirements, policy version, latency, token/usage data where available, result status, fallback count and validation result.

• Track cost by provider, model, agent, client and project.

• Track provider failure rate.

• Track fallback frequency.

• Track quality score.

• Track human escalation.

• Track model migrations and configuration changes.

• Never log raw secrets/API keys.

23. Admin Policies

• Global preferred providers.

• Per-agent preferred providers.

• Per-agent fallback chain.

• Maximum cost per task.

• Maximum monthly spend.

• Allowed data classes per provider.

• Minimum quality score.

• Maximum latency.

• Required model version/pinning.

• Canary percentage.

• Human approval requirement.

• Emergency disable rules.

24. Emergency & Degraded Mode

• If all preferred providers fail, AgencyOS enters DEGRADED_AI_MODE.

• Client-facing workflows should send safe, honest status messages rather than inventing results.

• Existing project state remains intact.

• Non-AI deterministic operations such as viewing invoices, project status and audit history should continue where possible.

• Admin receives an incident notification.

• When providers recover, queued safe tasks resume according to idempotency rules.

• No financial/destructive operation should be replayed blindly.

25. Provider Onboarding Contract

• Provider name and API endpoint.

• Authentication method.

• Model discovery method.

• Capabilities.

• Supported modalities.

• Context limits.

• Structured output/tool calling support.

• Pricing metadata.

• Rate limits.

• Health check.

• Error normalization.

• Streaming/batch support where relevant.

• Privacy/data policy metadata.

• Provider adapter version.

A provider is considered production-ready only after passing AgencyOS integration tests.

26. Routing Governance

> The Orchestrator may choose among approved models. It may not change the agency's business rules.

• Business policy belongs to Admin Policy Engine.

• Provider configuration belongs to AI Integration layer.

• Routing logic belongs to Orchestrator.

• Task execution belongs to specialized agents.

• Deterministic business validation belongs to application/backend logic.

• Final authority for governed actions belongs to Admin.

27. Recommended Initial Production Configuration

• Sales: OpenAI/Claude primary; Gemini/Grok fallback.

• PM: Claude/OpenAI primary; Gemini/DeepSeek fallback.

• UI: Gemini primary; OpenAI/Claude fallback.

• Prototype: OpenAI primary; Claude/Gemini fallback.

• Development: Claude/OpenAI primary; DeepSeek cost-optimized fallback.

• QA: Claude/OpenAI primary; Gemini/DeepSeek fallback.

• Finance: OpenAI/Claude with deterministic backend validation; Gemini fallback.

• Support: fast OpenAI/Gemini tier; Claude escalation.

• Customer Success: OpenAI/Gemini; Claude escalation.

• Upsell: OpenAI/Claude; Gemini/Grok fallback.

• Orchestrator: strongest approved reasoning tier; independent fallback provider.

These are starting recommendations. Admin must be able to change them without redeploying the core application.

28. Acceptance Criteria

• At least five provider integrations can be configured independently.

• API keys are securely stored and masked.

• Models can be enabled/disabled.

• Agent-to-provider assignments are configurable.

• Fallback chains are configurable.

• Provider health is visible.

• Usage/cost is measurable.

• Routing decisions are auditable.

• Model output is validated before workflow state changes.

• A provider failure can automatically route to an approved fallback.

• No agent contains hard-coded dependency on a single vendor.

• Admin can change routing without code changes.

• Production-critical workflows can use pinned model versions where required.

• Model discovery/version changes do not break business logic.

29. Final Architecture

> AGENT TASK → ORCHESTRATOR → CAPABILITY MATCH → POLICY → PROVIDER HEALTH → COST/QUALITY → MODEL SELECTION → EXECUTION → OUTPUT VALIDATION → SUCCESS / FALLBACK → AUDIT → NEXT WORKFLOW STEP

The result is an AgencyOS that can use Gemini, OpenAI/ChatGPT, Claude, Grok, DeepSeek and future providers as interchangeable AI infrastructure. The agency chooses the business rules; the Orchestrator chooses the best approved model for the job.

> The goal is not 'use every AI'. The goal is 'use the best available approved AI for each job, while remaining reliable, affordable, auditable and replaceable.'

30. External Reference Note

Provider model catalogs, pricing and capabilities change frequently. Implementation must verify current provider metadata during integration and should not hard-code today's model names as permanent business rules.

Current official documentation confirms that provider APIs expose model/capability information and that model names can change. DeepSeek documents V4-Pro/V4-Flash and migration from older aliases; xAI documents dynamic model listing, aliases and pricing metadata. ■cite■turn0search0■turn0search1■turn0search2■